GUÍA EXPLICATIVA DEL CÓDIGO - ASIGNACIÓN INDIVIDUAL POR MÓDULOS

Proyecto: Sangre y Dogma: Arena Elemental (C# / SharpDevelop 5 / Windows Forms)

Institución: IUJO Barquisimeto | Materia: Algoritmos y Programación I (ALP-265) - Prof. Pablo Lucena.

Equipo de Defensa: 1. Fabián | 2. Deibislin | 3. Jesús | 4. Yeferson | 5. Ereu

Objetivo: Guía didáctica para que cada estudiante defienda con total seguridad su módulo asignado ante el profesor y el auditor, abarcando el 100% del código, la arquitectura, los datos y el manejo de imágenes.

==========

MÓDULO 1: ARQUITECTURA GENERAL, MENÚS Y CICLO DE VIDA (DEFENDIDO POR: FABIÁN)

==========

Archivos asignados: Program.cs, PantallaInicioForm.cs, MainForm.cs, FormLore.cs

1. Punto de Entrada y Flujo de Pantallas (Program.cs y PantallaInicioForm.cs):

- Program.cs es el punto de inicio de la aplicación en el método Main(), ejecutando Application.Run(new PantallaInicioForm()).

- PantallaInicioForm muestra la portada (portada.jpg) en un PictureBox (picPortada) con SizeMode = Zoom. Al hacer clic en 'Iniciar Juego', abre MainForm con ShowDialog() y al cerrar regresa ordenadamente.

2. Menú Principal y Sistema Bilingüe a Nivel Estudiante (MainForm.cs):

- Para cumplir la exigencia de mantener el sistema bilingüe sin usar librerías complejas ni archivos .resx, se utiliza una variable global public static string Idioma = "ES"; y se cambia el texto de los botones con condicionales básicos if/else:

```
public static string Idioma = "ES";
private void AplicarIdioma() {
if (Idioma == "ES") { btnLore.Text = "MOSTRAR HISTORIA"; btnSalir.Text = "Salir"; }
else { btnLore.Text = "MOSTRAR HISTORIA / SHOW STORY"; btnSalir.Text = "Exit"; }
}
```

- Permite alternar entre 'ES' y 'EN' con el botón BtnCambiarIdioma, actualizando al instante toda la pantalla.

3. Botón 'MOSTRAR HISTORIA' y Visor de Historias (FormLore.cs):

- El auditor exigió quitar lore filosófico excesivo y dejar biografías breves de contexto y motivo. En FormLore se listan los 12 personajes jugables y 4 escenarios. Al pulsar 'MOSTRAR HISTORIA', muestra su historia individual y ficha técnica.

- En el PictureBox picCarta se carga la carta en alta resolución mediante CalculadoraCombate.ObtenerImagenCartaGrande(p).

==========

MÓDULO 2: MODELADO ORIENTADO A OBJETOS Y POLIMORFISMO (DEFENDIDO POR: DEIBISLIN)

==========

Archivo asignado: Personaje.cs (Clases Personaje, Protagonista, Duende, CartaEscenario)

1. Encapsulamiento y Separación de Arquitectura:

- Todos los atributos de la clase base Personaje son privados (id, nombre, elemento, hp, hpMax, def, atk, ener, esp, emp, historia) y se accede a ellos mediante propiedades públicas get y set.

```
private int hp; private int def; private int atk;
public int HP  { get { return hp; }  set { hp = value; } }
public int DEF { get { return def; } set { def = value; } }
```

```
public int ATK { get { return atk; } set { atk = value; } }
```

- Corrección del Profesor: Se eliminó todo MessageBox.Show de Personaje.cs. La clase es un modelo de datos puro; toda interacción gráfica reside exclusivamente en los formularios (Forms).

- Regla de negocio de 2do semestre: En el constructor de cada personaje se valida que HP + DEF + ATK sume exactamente 100 puntos.

2. Herencia y Polimorfismo con virtual y override:

- Protagonista : Personaje hereda todas las propiedades base y agrega la propiedad Rol (Guerrero, Maga, Sanador, Alquimista, Brujo), redefiniendo EjecutarHabilidadEspecial() con override.

- Duende : Personaje hereda de Personaje y añade la propiedad Nivel (1 al 7), calculando su daño especial en base a su nivel: 10 + (Nivel * 3).

- CartaEscenario maneja los efectos de campo mediante AplicarBuffNerf() (+4 ATK al elemento afín, -2 DEF al resto) y DispararTerremoto() cuando la empatía elemental EMP >= 50.

==========

MÓDULO 3: ESTRUCTURAS DE DATOS, RELEVOS Y ARCHIVOS TXT (DEFENDIDO POR: JESÚS)

==========

Archivos asignados: GestorEstructuras.cs, FormGeneradorMazos.cs

1. Reconciliación de Estructuras (Contrato vs Enunciado):

- Arreglo Estático (Contrato): public static Personaje[] Catalogo12Personajes = new Personaje[12]; almacena el catálogo inicial fijo de 12 personajes divididos en los 4 elementos (Agua, Tierra, Viento, Fuego).

- Listas Dinámicas List<T> (Contrato): Se cargan los personajes con ciclos for y .Add(). En QuitarPersonajeDeMazo() se demuestra el borrado de elementos con mazo.Remove(elegido).

- Colas Queue<Personaje> (Enunciado): ColaJugador1 y ColaJugador2 administran los 5 personajes en combate bajo la regla FIFO (First-In, First-Out). El personaje en banca entra al final con .Enqueue() y sale a pelear con .Dequeue().

- Pila Dinámica Stack<string> (Enunciado): PilaHistorial apila cada turno y ataque con .Push() siguiendo la regla LIFO (Last-In, First-Out).

2. Validación de Entradas en FormGeneradorMazos.cs:

- Valida con int.TryParse que los valores ingresados en las cajas de texto sean enteros y comprueba con un if que hp + def + atk == 100 antes de permitir agregar la carta a la lista.

3. Persistencia de Datos en Archivos Planos (.TXT):

- GestorEstructuras.GuardarTxt(): Usa StreamWriter para volcar el historial de la partida a HistorialBatalla.txt. Cumpliendo la corrección del profesor, al terminar el combate inserta un salto de línea y el texto destacado:

```
sw.WriteLine("");
sw.WriteLine("================================================");
sw.WriteLine(">>> SE ACABÓ EL COMBATE - GANADOR: " + ganador + " <<<");
sw.WriteLine("================================================");
```

- RegistrarPuntajeMaximoTxt() y LeerPuntajesMaximosTxt() escriben y leen PuntajesMaximos.txt con StreamReader.

==========

MÓDULO 4: MOTOR DE COMBATE Y SISTEMA DE IMÁGENES/PICTUREBOX (DEFENDIDO POR: YEFERSON)

==========

Archivos asignados: FormCombate.cs, CalculadoraCombate.cs (Renderizado Gráfico)

1. Interfaz y Lógica del Combate por Turnos (FormCombate.cs):

- Sorteo inicial con Random aleatorio = new Random(); Turno = aleatorio.Next(1, 3);

- Economía de Energía (ENER): Atacar cuesta 1 ENER; si le queda energía puede encadenar otro ataque o pulsar Pasar Turno. Al cambiar de turno suma +1 ENER.

- Habilidad Especial (ESP): Se carga en una ProgressBar (0 a 2) durante 2 turnos y cuesta 2 ENER.

- Fin de Partida y Cierre: VerificarFinDePartida() detecta si la vida llega a 0 y la Cola está vacía. Inhabilita todos los botones de acción con Enabled = false, declara el ganador y regresa al menú principal con this.Close().

2. CÓMO SE AÑADIERON Y FUNCIONAN LAS IMÁGENES AL CÓDIGO (100% Detallado):

- Paso A (Inclusión en el Proyecto): Las 5 cartas de los protagonistas (prota_1.jpg a prota_5.jpg) y las 7 de los duendes (duende_nv1.jpg a duende_nv7.jpg) se agregaron al archivo .csproj con <CopyToOutputDirectory>PreserveNewest</CopyToOutputDirectory> para que se copien a bin/Debug.

- Paso B (Búsqueda de Rutas): BuscarRutaArchivo() usa AppDomain.CurrentDomain.BaseDirectory y Path.Combine para ubicar los archivos en cualquier PC sin rutas fijas.

- Paso C (Carga sin Bloqueo de Disco): CargarImagenSinBloqueo() abre un FileStream de solo lectura y crea un Bitmap en memoria RAM, evitando que Windows bloquee los archivos .jpg en disco.

```
using (FileStream fs = new FileStream(ruta, FileMode.Open, FileAccess.Read)) {
using (Image temp = Image.FromStream(fs)) { return new Bitmap(temp); }
}
```

- Paso D (Renderizado con GDI+ en CalculadoraCombate): GenerarImagenCarta(p) dibuja la ilustración en un lienzo de 130x170 con g.DrawImage(), le dibuja una franja inferior oscura con g.FillRectangle(), escribe en tiempo real 'HP: ... ATK: ... DEF: ...' con g.DrawString() y pinta el marco con el color del elemento.

- Paso E (Asignación en Controles): En ActualizarPantalla() se asigna picCartaJ1.Image y picCartaJ2.Image, cambiando automáticamente cada vez que se produce un ataque o un relevo con la Cola.

==========

MÓDULO 5: MATEMÁTICAS DE CÁLCULO, BASE DE DATOS Y REDES (DEFENDIDO POR: EREU)

==========

Archivos asignados: CalculadoraCombate.cs (Fórmulas), ConexionBD.cs, FormGestionBD.cs, FormCombateRed.cs

1. Lógica Matemática Basada en Cálculo (CalculadoraCombate.cs):

- Límite matemático para el multiplicador de efectividad elemental: Multiplicador = 1.0 + (EMP / 100).

- Fórmula de daño mitigado por defensa: Dano = (ATK * ATK) / (ATK + DEF).

¿Por qué esta fórmula? Si se usara una resta simple (ATK - DEF), cuando la defensa fuera mayor que el ataque el daño daría negativo (curando al rival). La división representa un límite matemático: a mayor defensa, el daño se aproxima asintóticamente a cero pero nunca se vuelve negativo.

2. Base de Datos SQL Relacional Normalizada y Parámetros (ConexionBD.cs):

- 3 Tablas normalizadas por Clave Primaria (PK) y Clave Foránea (FK): Jugadores (PK: IdJugador), Personajes (FK: IdJugador) y Estadisticas (FK: IdJugador).

- Consultas Parametrizadas: Para evitar inyecciones SQL y cumplir con buenas prácticas

básicas, todas las consultas usan SqlParameter en lugar de concatenar cadenas:

```
string sql = "INSERT INTO Jugadores (Nombre) OUTPUT INSERTED.IdJugador VALUES (@Nombre)";
SqlCommand cmd = new SqlCommand(sql, conexion);
cmd.Parameters.Add(new SqlParameter("@Nombre", nombreJugador));
```

- En FormGestionBD.cs se calculan Partidas Jugadas (Victorias + Derrotas) y el Porcentaje de Éxito ((Victorias / Partidas) * 100). Si SQL Server no está instalado, guarda automáticamente en RespaldoBD.txt.

3. Combate en Red LAN por Sockets (FormCombateRed.cs):

- Conecta dos ventanas o PCs usando TcpListener (Servidor - Jugador 1) y TcpClient (Cliente - Jugador 2) en el puerto 5000.

- Envía cadenas de texto simples separadas por comas (ej: 'ATAQUE,18,Golpe de Tierra') que se leen con StreamReader y se procesan linealmente con paquete.Split(',').